实验四报告

一、XP 开发方法总结

通过阅读《XP_Overview》心思想和工作模型理解如下：

1. XP 的基本价值观

沟通：强调面对面交流，减少文档依赖，通过结对编程、代码规范等促进信息流动。

简单：始终做“最简单的可行方案”（DTSTTCPW），不提前设计复杂功能。

反馈：通过单元测试、功能测试、频繁发布获得即时反馈，指导后续开发。

勇气：敢于重构代码、拥抱需求变化、放弃无用功能。

2. 四个控制变量

成本、时间、质量、范围。客户可以控制其中三个，开发团队控制剩余一个。时间为核心变量，范围最常被调整。

3. 过程模型（Planning Game）

探索阶段：客户编写“用户故事”（User Story），开发团队估算时间（理想工程日）。

规划阶段：根据团队速度（负载因子）和发布日期，客户选择本次发布包含的故事。

指导阶段：通过迭代开发、恢复（重新协商）、新增故事、重新估算来动态调整计划。

4. 开发迭代

每个迭代 2~3 周，开始前进行迭代计划游戏，将故事拆解为工程任务（Engineering Task）。任务由开发者自愿认领并估算，形成迭代计划。

5. 核心实践

持续集成：每天多次集成代码，保证系统始终可运行。

集体代码所有权：任何开发者可修改任何代码，消除瓶颈。

结对编程：两人一组编写生产代码，一人编码一人复审，知识快速传递。

编码标准：统一命名、格式，便于集体维护。

现场客户：真实客户常驻团队，随时澄清需求、编写验收测试。

relentless testing：单元测试（开发人员写）和功能测试（客户写）自动化，测试先行。

6. 设计实践

不进行预先的大设计，依靠重构不断优化结构。

系统隐喻（System Metaphor）提供统一的命名和架构意象。

延迟优化（Lazy Optimization），先让代码工作、正确，再考虑性能。

小结：XP 是一种轻量、敏捷、拥抱变化的开发方法，特别适合需求不稳定、规模较小（2~12 人）的团队。它通过极致的实践组合（测试驱动、结对编程、持续集成等）来降低风险，提高软件质量与客户满意度。

二、DevOps 总结

阅读《devops》我了解到：

1. 定义与起源

DevOps 由 Patrick Debois 于 2009 年提出，是 Development（开发）与 Operations（运维）的组合。它是一种文化、运动和实践，强调打破开发与运维之间的“混乱之墙”，促进协作、自动化和持续交付。

2. 为什么需要 DevOps：

（1）开发追求快速变更，运维追求稳定可靠，二者目标冲突导致延迟、错误。

（2）传统模式下环境不一致、手动部署、配置管理混乱、生产事故频发。

（3）DevOps 通过统一工具链、共享责任、自动化流程来解决这些问题。

3. 核心原则（CALMS 模型）

Culture：文化，消除指责，鼓励开放的事后分析和奖励失败中的学习。
Automation：自动化，包括基础设施即代码、持续集成、持续部署、持续测试、持续监控。
Lean：精益，聚焦业务价值，消除浪费。
Measurement：度量，跟踪性能指标、日志、业务 KPI。
Sharing：共享，跨团队协作，共同目标。

4. 量化收益（调查数据）

提升部门间协作（23%）、提高部署质量（22%）、增加客户使用量（22%）、缩短上市时间（20%）等。

5. 7Cs 框架（Happiest Minds）

沟通、协作、受控过程、持续集成、持续部署、持续测试、持续监控。其中自动化是核心。

6. 主要障碍

缺乏文化、工具不兼容、技能不足、管理层不支持等。

小结：DevOps 不仅是工具链的集成，更是一种文化和流程变革。它使软件交付更快速、稳定、可靠，是实现持续交付的关键路径。

三、活动图练习题

练习题 2（图 3-23）

题目描述：

2. 图3-23是一个软件开发项目的活动图。对应于图中每条边的数字表示完成这条边代表的活动所需的天数。例如，完成终止于里程碑E的活动需要4天时间。对于每个活动，列出它的前驱，并计算最早开始时间、最晚开始时间和时差。然后确定出关键路径。

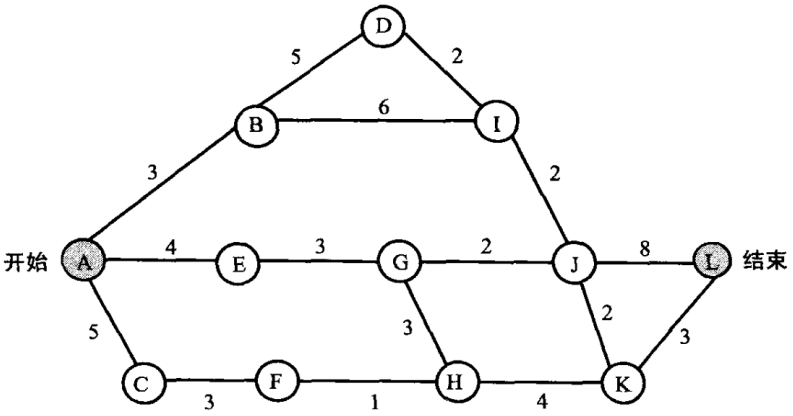


图3-23　练习2的活动图

活动列表及前驱

活动	持续时间	前驱活动
A→B	3	无
A→E	4	无
A→C	5	无
B→D	5	A→B
B→I	6	A→B
D→I	2	B→D
I→J	2	B→I, D→I
E→G	3	A→E

G→J	2	E→G
C→F	3	A→C
F→H	1	C→F
G→H	3	E→G
H→K	4	F→H, G→H
J→K	2	I→J, G→J
J→L	8	I→J, G→J
K→L	3	H→K, J→K

各活动的最早开始、最晚开始、时差

活动	最早开始(ES)	最晚开始(LS)	时差(LS-ES)
A→B	0	0	0
A→E	0	3	3
A→C	0	4	4
B→D	3	3	0
B→I	3	4	1
D→I	8	8	0
I→J	10	10	0
E→G	4	7	3
G→J	7	10	3
C→F	5	9	4
F→H	8	12	4
G→H	7	10	3
H→K	10	13	3
J→K	12	15	3
J→L	12	12	0
K→L	14	17	3

关键路径

时差为 0 的活动构成关键路径：**A→B → B→D → D→I → I→J → J→L**

总工期 = 3 + 5 + 2 + 2 + 8 = **20 天**

练习题 3（图 3-23）

3. 图3-24是一张活动图。找出其关键路径。

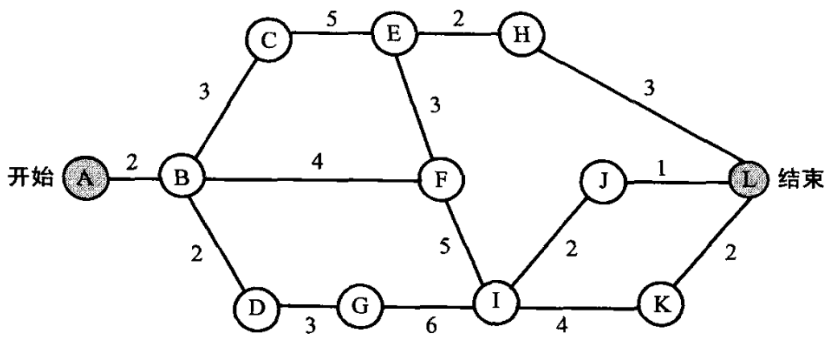


图3-24　练习3的活动图

根据图 3-24 的结构（节点：开始、A、B、C、D、E、F、G、H、I、J、K、结束；边及持续时间：开始→A:5，开始→B:3，开始→C:4，A→D:2，B→D:3，C→E:1，D→F:2，E→F:2，F→G:3，F→H:6, G→I:5, H→I:4, I→J:2, J→K:1, K→结束:3），计算所有路径的总时间：

路径	**总时间**
开始→A→D→F→G→I→J→K→结束	5+2+2+3+5+2+1+3 = 23
开始→A→D→F→H→I→J→K→结束	5+2+2+6+4+2+1+3 = **25**
开始→B→D→F→G→I→J→K→结束	3+3+2+3+5+2+1+3 = 22
开始→B→D→F→H→I→J→K→结束	3+3+2+6+4+2+1+3 = 24
开始→C→E→F→G→I→J→K→结束	4+1+2+3+5+2+1+3 = 21
开始→C→E→F→H→I→J→K→结束	4+1+2+6+4+2+1+3 = 23

最长路径长度为 **25 天**，对应的关键路径为：**开始 → A → D → F → H → I → J → K → 结束**。

四、小组项目（博客系统）工作活动分解与活动图

1. 项目背景与开发模型

本小组项目为“BlogZone（分区可配置的网络博客系统）”，采用 Scrum 敏捷开发方法，共规划 3 个 Sprint，每个 Sprint 周期为 2 周（10 个工作日）。项目整体功能规划采用“必做核心闭环 + 迭代增强扩展”的双层策略（详见《可行性分析报告》第 6 章）。

Sprint1（MVP 核心）：用户注册登录、分区浏览、图文发帖、点赞评论、建区申请与后台治理基础功能。

Sprint2（治理与生长）：建区申请审核闭环、内容下架/置顶、审计日志、VIP 模拟、搜索、社交互动（转发/收藏/关注）。

Sprint3（扩展与优化）：亮点模块（问答/二手/活动选一）、分区模板、二级分区自治、性能优化。

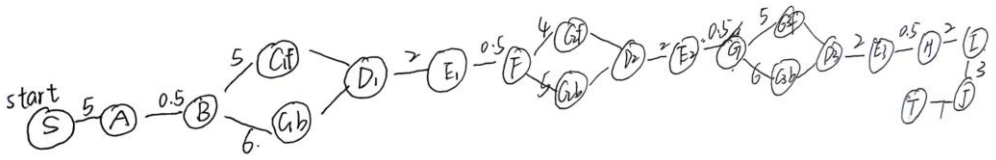
2. 工作活动分解与持续时间估算

为绘制类似练习题 2、3 风格的活动图（节点表示事件，边表示活动），我们将项目主要工作抽象为以下活动，并基于团队规模（4 人）和并行开发原则估算各活动的持续时间（天）：

活动编号	活动名称	持续时间（天）	前驱活动
S→A	项目初始化与需求细化	5	无
A→B	Sprint1 规划会议	0.5	S→A
B→C1f	Sprint1 前端开发（用户认证、分区浏览、发帖界面）	5	A→B
B→C1b	Sprint1 后端开发（认证接口、分区 API、发帖、互动、建区申请）	6	A→B
C1f, C1b→D1	Sprint1 测试与修复	2	C1f, C1b
D1→E1	Sprint1 评审与复盘	0.5	D1
E1→F	Sprint2 规划会议	0.5	E1
F→C2f	Sprint2 前端开发（社交互动页面、搜索界面、VIP 展示）	4	E1→F
F→C2b	Sprint2 后端开发（审核流程、搜索接口、通知、审计日志）	5	E1→F
C2f, C2b→D2	Sprint2 测试与修复	2	C2f, C2b
D2→E2	Sprint2 评审与复盘	0.5	D2
E2→G	Sprint3 规划会议	0.5	E2
G→C3f	Sprint3 前端开发（亮点模块界面、分区模板配置）	5	E2→G
G→C3b	Sprint3 后端开发（亮点模块逻辑、二级分区自治、性能优化）	6	E2→G
C3f, C3b→D3	Sprint3 测试与修复	2	C3f, C3b
D3→E3	Sprint3 评审与复盘	0.5	D3
E3→H	系统集成测试	2	E3
H→I	文档编写与验收准备	3	H
I→J	项目总结	1	I

3. 项目活动图

下图为手绘活动图



4. 关键路径计算

项目总工期为从 S 到 T 所有路径中的最长路径长度。计算各条路径：

路径 1（经过 C1b、C2b、C3b）：

S→A(5) + A→B(0.5) + B→C1b(6) + C1b→D1(0) + D1→E1(2) + E1→F(0.5) + F→C2b(5) + C2b→D2(0) + D2→E2(2) + E2→G(0.5) + G→C3b(6) + C3b→D3(0) + D3→E3(2) + E3→H(0.5) + H→I(2) + I→J(3) + J→T(1)

= 5 + 0.5 + 6 + 2 + 0.5 + 5 + 2 + 0.5 + 6 + 2 + 0.5 + 2 + 3 + 1 = **36 天**

路径 2（经过 C1b、C2b、C3f）：

将上路径中的 G→C3b(6)换为 G→C3f(5)，则总长为 35 天。

路径 3（经过 C1b、C2f、C3b）：

将 F→C2b(5)换为 F→C2f(4)，总长为 35 天。

路径 4（经过 C1f、C2b、C3b）：

将 B→C1b(6)换为 B→C1f(5)，总长为 35 天。

其他组合路径更短。

最长路径为 36 天，对应 **路径 1**：

S→A→B→C1b→D1→E1→F→C2b→D2→E2→G→C3b→D3→E3→H→I→J→T。

关键路径上的活动为：

项目初始化 → Sprint1 规划 → **Sprint1 后端开发** → Sprint1 测试 → Sprint1 评审复盘 → Sprint2 规划 → **Sprint2 后端开发** → Sprint2 测试 → Sprint2 评审复盘 → Sprint3 规划 → **Sprint3 后端开发** → Sprint3 测试 → Sprint3 评审复盘 → 系统集成测试 → 文档验收 → 项目总结。

项目最短工期 = 36 天（约 7.2 周），与可行性分析中 3 个 Sprint（每 Sprint2 周）加前后准备收尾的预期基本吻合。

5. 小组协作与文档保存

小组采用 **GitHub** 作为协作平台：

冲刺待办列表：采用看板形式（GitHub Projects），实时更新任务状态。

文档版本管理：所有文档（包括可行性报告、设计文档、活动图等）均通过 Git 进行版本控制，保存在 GitHub 仓库的 docs/ 目录下。

进度跟踪：每次实验课结束更新。